

1. Problem framing and dataset analysis

The dataset and its context

The project uses the New York City (NYC) Yellow Taxi January 2019 trip dataset, which contains 7,696,617 raw taxi trip records from `yellow_tripdata_2019-01.parquet`. Each from this dataset record represents an individual taxi journey, it includes details such as pickup and dropoff timestamps, passenger count, trip distance, fare values, payment type, and pickup/dropoff location IDs. To make the location IDs meaningful, the trip data is joined with the NYC taxi zone lookup dataset, which maps each zone ID to its specified borough, zone, and service zone. This turns the starting raw dataset from a mere table of trip transactions into an urban mobility dataset that can be loaded to the database and can support questions about where people travel, which zones receive the most dropoffs, and how trip distance varies by time of day.

Data challenges

The main data challenges were not in the compulsory trip fields, but rather in the operational irregularities around them: required fields such as pickup time, dropoff time, trip distance, fare amount, total amount, and location IDs were present, while optional fields like `passenger_count`, `RatecodeID`, `store_and_fwd_flag`, `congestion_surcharge`, and especially `airport_fee` were missing for numerous records. Missing optional surcharge or metadata fields were tolerated, but records with invalid timestamps or negative fare values were removed because they would disfigure calculations of duration, speed, revenue. The dataset also contained anomalies such as trips outside January 2019, zero-distance trips, extremely long durations, unusually high fare totals, and trips showing speeds above 120 mph threshold. Instead of deleting these unusual trips, the pipeline flags them as suspicious using reasons like `unrealistic_speed`, `very_long_duration`, and `extreme_total_amount`, giving room for the dashboard to preserve potentially meaningful edge cases while still giving warnings for the analysis about them. Another challenge appears during joining of geographic and trip records. Some pickup and dropoff location IDs did not resolve cleanly to zone names, so at the analytics layer it handles unknown zones rather than assuming every trip can be mapped perfectly.

Assumptions during the cleaning process

Our cleaning logic assumes that a taxi trip is only analytically usable if its required movement and money fields are present and valid. pickup/dropoff timestamps, pickup/dropoff location IDs, trip distance, fare amount, and total amount must exist, the dropoff time must be after the pickup time, and distance/fare values cannot be negative. Records that violate these rules are removed because they would directly disrupt the derived values we depend on later, especially `trip_duration_minutes`, `average_speed_mph`, `fare_per_mile`, revenue summaries, and hourly distance trends. On the other hand, the outlier detector assumes that unusual trips are not automatically wrong, so it keeps them in the dataset but flags them with `is_outlier` along with a reason for that; this fits the TLC's own warning that trip records are collected from technology providers and are not guaranteed by TLC to be perfectly accurate or complete (New York City Taxi and Limousine Commission [NYC TLC], n.d.-a). The `MAX_DISTANCE_MILES = 100` threshold we implemented is suitable because normal NYC yellow taxi movement is usually local or airport-oriented, while 100 miles is far beyond the ordinary city travel and even exceeds most practical NYC-to-near-suburb taxi use, so it catches trips that may represent GPS, meter, or timestamp problems without deleting some rare negotiated out-of-city trips. The `MAX_TOTAL_AMOUNT = 500` and `MAX_FARE_AMOUNT = 500` thresholds as well are intentionally generous: TLC fare rules show that normal metered trips, JFK flat-rate trips, Newark trips, tolls, surcharges, and tips can legitimately raise a fare, but values above \$500 are far outside the expected passenger fare context and should be reviewed rather than trusted blindly (NYC TLC, n.d.-b). The `MAX_SPEED_MPH = 120` threshold is there because it is much higher than ordinary NYC street speeds and even above common highway limits, meaning we avoid over-flagging short trips affected by timestamp rounding while still catching physically unrealistic average speeds. Finally, `MAX_DURATION_HOURS = 8` flagging threshold treats a taxi trip lasting longer than a typical work shift as suspicious, which is acceptable because such records can heavily distort average duration and speed calculations even if the distance and fare fields appear valid.

unexpected observations that influenced the design

One unexpected observation, as tried to mention earlier, was that not every pickup and dropoff location ID could be matched to a readable taxi zone name during joining the geographic and trip records. After joining the cleaned trips with the taxi zone lookup table,

there were still around 157,903 unmatched pickup zones and 143,224 unmatched dropoff zones. This directly influenced the design of the analytics layer, now instead of dropping those trips or letting blank zone names appear in the charts, the top dropoff zone query uses `COALESCE(dropoff_zone, 'Unknown')` so unmatched records are still counted and represented honestly. This design choice allows to protect the full trip volume while at the same time making the data quality limitation visible to the user. This system was designed to tolerate defective and incomplete geographic records rather than hiding it.

2. System Architecture

Overview

The Urban Mobility Data Explorer is a system which has three-tiers: First, a Python ETL(Extract, Transform, Load) pipeline. This pipeline transforms raw New York City(NYC) Taxi and Limousine Commission (TLC) trip data into a queryable database. Second, a FastAPI framework for the backend which serves analytical endpoints, and third, a JavaScript used for the frontend to visualize the results through an interactive user interface dashboard.

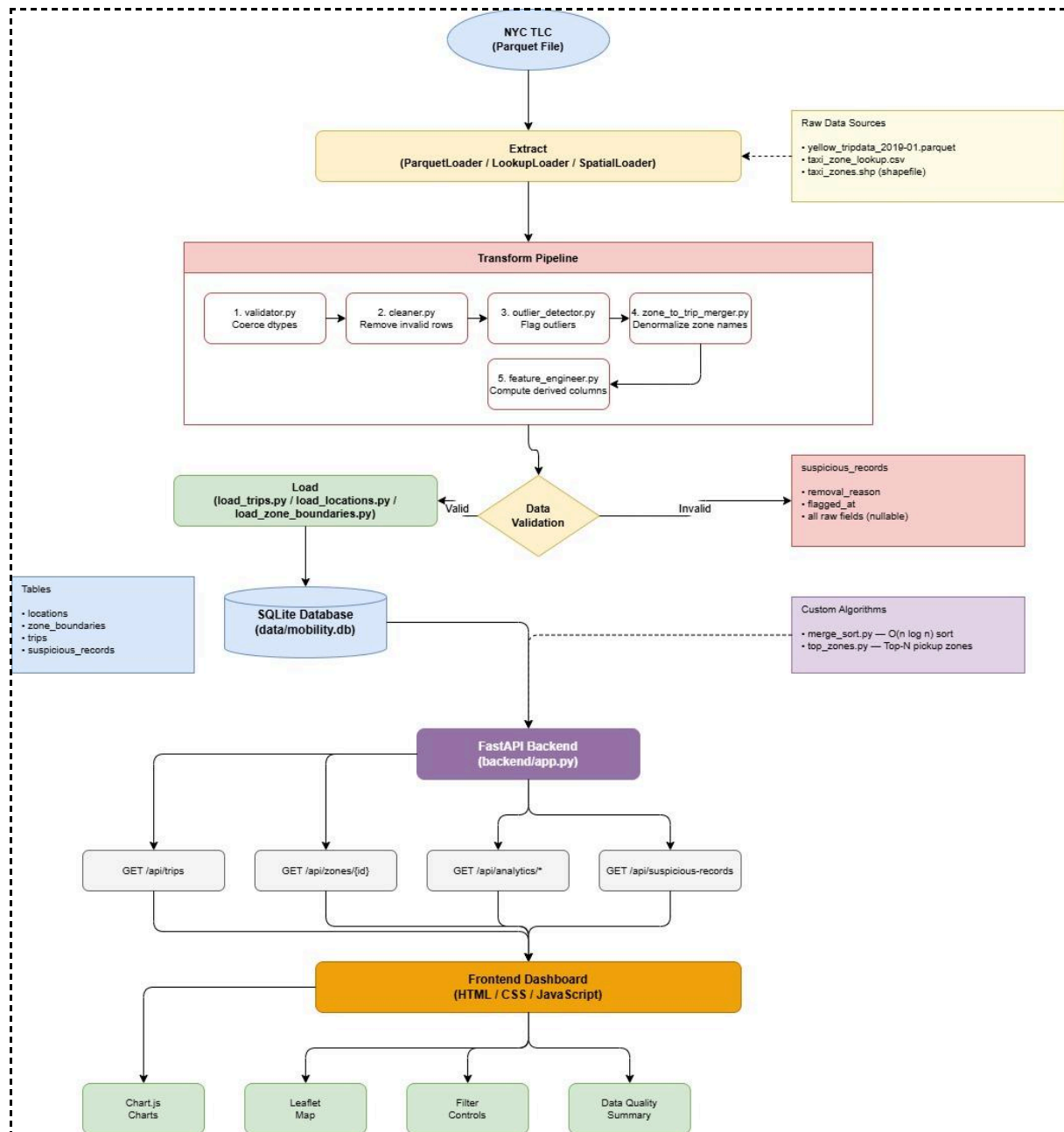


Fig 1. System Architecture Diagram

ETL Layer

The ETL pipeline runs in five logical stages, each one of them is implemented as an independent class with a single responsibility of what it does:

1. **Extract** (etl/extract/): The ParquetLoader, LookupLoader, and SpatialLoader read all the three raw files which serves as the data source and validate that all the required columns are present before any data leaves the file.

2. **Validate** (etl/transform/validator.py): Here all raw data types are coerced (strings to datetimes, floats to nullable integers) and reports every value that had to be coerced or was missing, without dropping any rows yet.
3. **Clean** (etl/transform/cleaner.py): At this stage, all the rows with missing required fields, invalid timestamps, negative distances or fares, or duplicates are removed. Every removed row is preserved with its original values plus a `removal_reason`, rather than being silently discarded.
4. **Outlier Detection** (etl/transform/outlier_detector.py): This flags (but it does not remove) trips with extreme distance, fare, speed, or duration. This distinction matters: a \$300 airport fare is suspicious, but it is still a real trip that should count toward revenue totals, other than a negative fare, which is a data error.
5. **Zone Merge + Feature Engineering** (zone_to_trip_merger.py, feature_engineer.py): This joins borough/zone names directly onto each trip row, and derives `trip_duration_minutes`, `average_speed_mph`, `fare_per_mile`, `pickup_hour`, `pickup_day_of_week`, and `tip_percentage`.

Because the data source file contains a very large amount of data that is about 7.6 million rows, the entire pipeline processes data in chunks of 50,000 rows instead of loading the full dataset into memory at once. Each chunk passes through all five stages before the next chunk is read.

Database Layer

For this project, we decided against using PostgreSQL or MySQL, etc, and instead used SQLite for three main reasons:

- No set up friction between a 3-person team. Each and everyone of the team members needed to run the full pipeline locally and independently. A file-based database that required no installation, configuration, or troubleshooting of a database server before building against real data.
- Good performance at this level with adequate indexing. SQLite is very effective at handling read intensive analytical queries (aggregations, GROUP BY, filtering) with millions of rows in the database as long as the appropriate columns are indexed. This project is not designed to be a project where many processes write at the same time, which is the case where SQLite's limitations actually would matter.

- No need for true spatial query functions! PostGIS is the primary motivation to use PostgreSQL here, for queries such as "which zone does this GPS point occur in?". It's no use to this project to have the added operational complexity of PostgreSQL, since we aren't doing anything that requires a spatial join.

One of the trade-offs of schema design is denormalization for query speed

The trips table is designed so that pickup_borough, pickup_zone, dropoff_borough, and dropoff_zone are text columns, and that the data is replicated in the locations table. This is not a mistake, but rather a conscious deviation from the norm.

The other option, to join to the other location in each subsequent analytical call, would add a join cost to each individual dashboard request to a table that had around 7.6 million rows.

This system is read-heavy – the same few aggregation queries are run repeatedly on the dashboard, and data is loaded once through the ETL system, so the cost of the join is paid up front while the dashboard can be a single-table query. pu_location_id and do_location_id are still foreign key references into locations, so referential integrity is maintained with the denormalized columns.

Backend Layer

We decided to go with FastAPI for the following practical reasons: Interactive API documentation is automatically generated when running the app (URL /docs), which enabled every member of the team to send requests without needing to write curl commands and install Postman; using Pydantic to validate requests before they even hit the database code; it is built on async, which gives us the possibility to scale the API as the dataset grows.

Routes are grouped by resources (trips, zones, suspicious_records, analytics) and each of the routers is responsible for one piece of the data model. The analytics router exposes pre-aggregated endpoints that are ready to be displayed on dashboards instead of the frontend having to download raw trip data and perform the aggregation themselves (which would be made of JavaScript logic). Keeping aggregation logic in SQL, near the data, rather than in JavaScript.

Frontend Layer

The front end is implemented using vanilla JavaScript and Chart.js, not a framework like React, as the project's scope (a handful of dashboard pages each fetching and charting data) did not warrant build tooling. All of the backend communication and chart-default styling is

centralized in a shared api.js module, so each page (Revenue, Mobility, Main Dashboard, Zones) has one consistent data-fetching layer and one consistent visual theme, instead of each page reimplementing both.

Key Trade-offs Summary

Decision	Alternative considered	Why this choice
SQLite	PostgreSQL	No team server setup required, just enough servers for analytics at this scale which are read-heavy.
A trip containing denormalized zone columns.	Relational databases and strict normalization using joins.	Eliminates a join on all analytical queries against 7.6M rows
FastAPI	Flask	Automatic documents for endpoint testing across the team; built-in validation.
ETL processing of chunks (50k rows)	Loading full dataset into memory	The number of rows in this table would not be able to fit in memory for an average laptop.
Vanilla JS + Chart.js	React	Project scope did not justify framework/build tooling overhead

3. Algorithmic Logic and Data Structures

A major analysis challenge in the Urban Mobility Data Explorer is determining the most active pickup areas in NYC. This information enables users to appreciate where there is a high requirement for taxis and can be used for mobility analysis between boroughs.

A custom Merge Sort algorithm was designed and implemented in the analytics module. The algorithm sorts pickup zones by the number of trips that start from a specific zone.

The process can be broken down into four steps:

1. Count the number of trips from each pickup area using a query.
3. Print the list of dictionaries.
3. Sort the list with the Merge Sort algorithm
4. Return the top N pickup zones to be shown on the dashboard.

Pseudocodes

FUNCTION FindTopPickupZones(topN) Count trips per pickup zone Sort zones by trip count using Merge Sort Take the top N zones Return result END FUNCTION	FUNCTION MergeSort(list) IF list contains 0 or 1 item RETURN list Split list into two halves Sort left half using MergeSort Sort right half using MergeSort Merge the two sorted halves into one sorted list Return merged list END FUNCTION	FUNCTION Merge(leftList, rightList) Create empty result list WHILE both lists contain items Compare the first item from each list Add the larger value to the result list Add any remaining items from left list Add any remaining items from right list Return result list END FUNCTION
---	---	--

Time and Space Complexity

Operation	Complexity	Where
Counting pickup zones	$O(n)$	N = Total trips taken
Merge Sort	$O(z \log z)$	Z = number of distinct pick-up areas
Top_n extraction	$O(k)$	K = desired number of top zones

Why Merge Sort?

The choice of Merge Sort has been made since it guarantees $O(n \log n)$ time when sorting any data. It also requires algorithmic thinking and knowledge of divide-and-conquer methods.

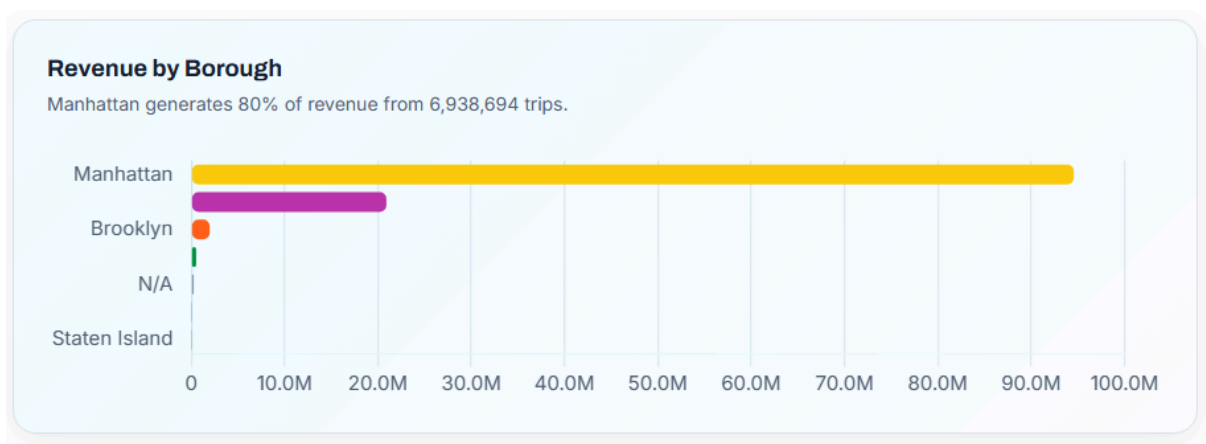
The algorithm is in use in the endpoint **GET /api/analytics/top-pickup-zones**, which pre-sorts the pickup zones before returning them to the dashboard.

4. Insights and Interpretation

Insight 1: Manhattan dominates volume but not value per trip

It's recorded by having the **/api/analytics/revenue-by-borough** endpoint to query the trips table, and group them by pickup_borough and summing total_amount. The result was visualized as a horizontal bar chart on the Revenue Dashboard.

The key finding is that Manhattan generated \$93,909,946 in total revenue from 6,926,632 trips in January 2019, and it represents about 90% of all trips and 80% of all revenue in the dataset. But then, the Manhattan's average fare of \$13.56 per trip is the lowest of all boroughs.



Interpretation: This reveals a very important pattern and an interesting one at that about urban taxi economics. Manhattan's revenue dominance comes entirely from volume, not fare value. The borough's density means most trips are short, crosstown rides, quick downtown hops, last-mile connections, each of these locations trips generates modest fares but occurring at such frequency that the cumulative revenue made other boroughs shorter. It suggests that taxi demand in Manhattan is driven by convenience and proximity rather than distance, and is where the influence of the increased average fare (such as surge pricing or minimum fare rules) would face significant passenger resistance in a market built on short, predictable trips.

Insight 2: Airport zones command the highest average fares despite minimal trip volume

This was recorded by having the `/api/analytics/average-fare` endpoint breaks down average fare, tip, and total by borough and payment method. EWR (Newark Airport) showed the highest average total per trip across all boroughs.

The key findings shows that the EWR generated only 439 trips in January 2019, the lowest volume of any borough, but still it has an average fare of about \$94.58 per trip which is nearly 7 times Manhattan's average. Queens, which includes JFK and LaGuardia airports, showed an average of \$44.51 per trip on 464,635 trips.

Revenue Ranking		
Top pickup zones by total revenue		
1	JFK Airport Queens · 195.7K trips	\$11,144,991
2	LaGuardia Airport Queens · 183.9K trips	\$7,570,737
3	Midtown Center Manhattan · 312.4K trips	\$4,442,846
4	Upper East Side South Manhattan · 332.3K trips	\$4,429,897
5	Times Sq/Theatre District Manhattan · 263.6K trips	\$4,003,129
6	Midtown East Manhattan · 277.1K trips	\$3,867,272
7	Upper East Side North Manhattan · 322.9K trips	\$3,796,674
8	Penn Station/Madison Sq West Manhattan · 260.5K trips	\$3,692,370
9	Murray Hill Manhattan · 239.3K trips	\$3,295,318
10	Clinton East Manhattan · 240.9K trips	\$3,234,620

Interpretation: NYC TLC flat-rate rules for airport trips (a fixed \$52 fare for JFK plus tolls and surcharges, and negotiated rates for Newark) mean that airport zones are different from metered city trips in a structural way. It shows that one EWR trip makes almost roughly the same amount of money as seven Manhattan trips can make. It has very direct implications for fleet allocation, drivers seeking higher per-trip revenue are focused more on the airport zones, which can create scarcity of drivers in most high-volume city zones during peak airport hours. The data validates a well-known tension in taxi economics between per-trip revenue optimization (airport trips) and total daily earnings through volume (Manhattan trips).

Insight 3: Daily revenue follows a clear weekly cycle with identifiable anomalies

This was recorded using the `/api/analytics/revenue-trends` endpoint groups total revenue by `pickup_datetime` date using SQLite's `SUBSTR()` function to extract the date portion, returning one row per day across January 2019. This was visualized as a line chart on the Revenue Dashboard.

The key findings shows that the daily revenue across January 2019 is showing a consistent weekly rhythm whereby revenue dips noticeably on weekends especially on Sundays and recovers through the working weekday. January 1st (New Year's Day) shows a significant spike above the weekly baseline, while January 20th (Martin Luther King Jr. Day, a federal holiday) shows a mid-week dip inconsistent with the surrounding pattern.

Interpretation: The weekly cycle confirms that NYC yellow taxi demand is strongly affected by the commuter and business travel patterns in the city. It shows that the taxi network serves a working city. The New Year's Eve spike reflects a predictable surge in demand when public transit feels less safe or convenient for late-night travel, while the Martin Luther King Jr. Day drop shows that federal holidays genuinely impact the business travel that drives weekday volume. For a real-world product, this pattern would directly inform dynamic pricing algorithms and driver shift recommendations, knowing that Sunday mornings and federal holidays consistently underperform allows operators to plan accordingly instead of assuming that everyday is just the same as others.

5. Reflection and Future Work

Technical challenges

We faced a lot of changes on this project but the most significant technical challenge was not in a single piece of code but in the coordination of each of the team members pipeline where each component depended on the others. The database schema had to be designed before the ETL could load data into it, but then again, the schema needed to accurately reflect what the pipeline would produce, such as the column names, data types, nullable fields, and derived feature names and to do that before the ETL code was finished. This required an early alignment conversation about column naming conventions (e.g., `trip_duration_minutes` rather than `duration` or `trip_duration`) that helps to save the silent mismatches problem later. A similar dependency existed between the loading scripts and the backend whereby the analytics endpoints could not be tested until the database was populated, and the frontend could not be tested until the endpoints returned real data. And to be very honest, managing

these sequential dependencies under a tight deadline required careful communication across the team about what was ready so that all of us can be on the same page, and know what was blocked, and what could be built against placeholder data in the meantime.

One major concrete technical difficulty we faced as a team working on this project, was the NA type incompatibility between pandas nullable integer and float types (introduced in newer pandas versions) and Python sqlite3 module, which accept only Python-native types. This came up repeatedly over and over again, at first during trip loading, then during analytics table population, because pandas Int64 and Float64 dtypes use pd.NA as their null sentinel rather than numpy.nan, and sqlite3 does not recognize pd.NA. The fix required a custom conversion function that clearly checks each value before insertion, handling numpy.nan, pd.NA, pd.NaT, numpy scalars, and datetime objects as separate cases.

The shapefile coordinate system presented another unexpected challenge. The taxi zone shapefile uses NAD83 State Plane New York coordinates measured in feet, a projected coordinate system optimized for surveying accuracy in New York, but instead the WGS84 latitude/longitude that web mapping libraries expect. Storing the raw coordinates would have made the frontend map completely unusable. This required reprojecting all 263 zone polygons to WGS84 using geopandas before storing them as GeoJSON strings in the database, adding a dependency on both geopandas and pyproj that had to be documented in requirements.txt for the rest of the team.

Team challenges

Coordinating three parallel workstreams on a shared repository introduced merge conflicts at several critical points, particularly in backend/app.py and frontend/components/sidebar.js, where each team member had made changes independently before pulling each other's updates. These merge conflicts nearly affected and broke into something serious for the team, but they were resolved through direct communication and careful manual merging, but they cost time that was supposed to be used to do something else. The experience reinforced that on a project with tight interdependencies, committing small and pulling frequently is more important than working in long isolated sessions. Which is more like working chunks, testing often and consistent communication, is very important.

What we would do differently

Given more time, there are many improvements to make and one of the most impactful ones we would have considered faster than others would be replacing the SQLite database with PostgreSQL and adding PostGIS for true spatial queries. The current system can display zone boundaries on a map but cannot answer spatial questions such as "which zone does this GPS coordinate fall in" or "which zones are adjacent to a given zone." These queries would enable trip origin/destination clustering and heat map visualizations that are currently impossible with a text-stored GeoJSON approach.

The ETL pipeline would benefit from proper idempotency, currently, running the loaders twice inserts duplicate records because there is no deduplication check on incoming data. A production system would use INSERT OR IGNORE with a natural key (trip timestamp + location IDs) or a proper upsert pattern to make pipeline re-runs safe.

The analytics cache system (pre-computed summary tables built from the full 7.6M trip dataset) was added late in development almost toward the submission deadline which required a separate refresh_analytics() step after the main data load. Trying to integrate and add this into the main pipeline at the final stage would make the setup process cleaner and reduce the risk of the dashboard showing no data because the analytics refresh was not run.

If this were a real-world product

The most meaningful extension would be Many month data, January 2019 captures one month of a single year. Seasonal patterns (summer tourism vs. winter commuting), year-over-year trend comparisons, and the impact of external events (weather, major events, transit disruptions) both positive and negative impact are all invisible with a single month's data. A real urban mobility platform would be more professional and presentable when rolling data from the TLC's monthly releases and maintain a historical database that grows over time, enabling the kind of long term analysis that would be genuinely useful to city planners and transportation researchers and many more.